A Systematic Literature Review of Large Language Models and Small Language Models in Hybrid, Multi-Agent, and Monolithic Architectures

Doğukan Topçu[1], Abdulbaki Çeltik[1], Berkhan Kargılı[1], Meriç Kelemeden[1], Onur Demirörs[2], and Hüseyin Ünlü[2]

[1] Izmir Institute of Technology, Izmir, Turkey
dogukantopcu@std.iyte.edu.tr
abdulbakiceltik@std.iyte.edu.tr
berkhankargili@std.iyte.edu.tr
merickelemeden@std.iyte.edu.tr
[2] onurdemirors@iyte.edu.tr
huseyinunlu@iyte.edu.tr

Abstract. Large language models (LLMs) now define the state of the art in many reasoning, generation, and decision-support tasks, but their deployment cost, energy demand, latency, and hardware requirements remain substantial. Small language models (SLMs) provide an attractive alternative for edge deployment, privacy-sensitive settings, and task-specific optimisation, yet their capabilities are uneven across domains. This paper presents a systematic literature review of 54 studies on LLMs, SLMs, and SLM–LLM collaboration, with emphasis on hybrid, multi-agent, retrieval-centered, and monolithic architectures. The review is guided by four research questions addressing comparative performance, computational efficiency, orchestration mechanisms, and domain suitability. Using a PRISMA 2020-guided review design we synthesize evidence on model choices, architectural patterns, datasets, metrics, baselines, latency, cost, energy, and reported limitations. The review shows a strong shift toward hybrid and multi-agent designs together with a clear reporting asymmetry in which accuracy is common but cost and energy are rarely quantified. The findings indicate that SLMs are most competitive when they are paired with routing, retrieval, planning, or fusion mechanisms and when the problem space is narrow, structured, or domain-specific.

Keywords: systematic literature review · large language models · small language models · hybrid architectures · multi-agent systems · edge-cloud inference

1 Introduction

Large language models (LLMs) excel at reasoning, coding, and multimodal tasks [1, 2, 3]. However, due to the high hardware requirements (RAM, GPU, etc.) they demand, these models are usually run only on cloud-based infrastructures or very advanced local systems [4, 5]. This creates a significant bottleneck for system developers seeking to reduce cloud dependency, ensure data privacy, and avoid high operational costs [6, 7].

As a result of these limitations, small language models (SLMs) are emerging as a strong alternative. Their ability to run on local machines provides a significant advantage in terms of data security and deployment autonomy, while also offering substantially lower operational costs than cloud-scale LLM setups [6, 7]. Moreover, recent studies show that SLMs can be adapted to specific domains

via fine-tuning or distillation, enabling competitive performance in specialized tasks [8, 9, 10]. In practical systems, SLMs are also attractive because they integrate well with explicit control modules (e.g., rule-based filters, routing/verification thresholds, or planners), improving controllability and reliability beyond prompt-only solutions [5, 7, 11]. However, SLMs still struggle to match the broad expertise and robust reasoning of massive frontier models, especially on tasks requiring deep contextual understanding and extensive world knowledge [3, 12]. Therefore, framing the problem as a simple "SLM-versus-LLM" competition is no longer sufficient; instead, current research investigates when a single SLM is adequate, when an LLM is necessary, and when hybrid or agent-based architectures are preferable [13, 14, 15].

The existing literature provides concrete, domain-specific examples showing that the choice between SLMs, LLMs, and hybrid designs is primarily a systems decision. In telecommunications, Symbiotic Agents reports up to a fivefold reduction in decision error and near-real-time operation with up to 99.9% lower GPU overhead using SLM variants [16]. In medicine, self-reflective pipelines pair a compact generator with an LLM-based judge to iteratively refine outputs and improve diagnostic reliability [17].

These developments provide practical alternatives for scenarios where relying on a single monolithic LLM is either insufficient for task demands or impractical under strict resource constraints. Accordingly, recent work increasingly targets measurable improvements beyond raw accuracy, including reduced computational load, lower operational cost, and tighter latency budgets [4,5,13,16]. Across domains, researchers have reported which hybrid design patterns are effective for specific task classes—for example, selective edge–cloud verification, expert routing, and retrieval- or planner-centered pipelines—and how these choices translate to real-world deployments [11, 13, 18, 19].

Motivated by these trends, this review analyzes when a single SLM is sufficient, when an LLM remains necessary, and when hybrid or multi-agent architectures provide the best trade-off between capability and efficiency. We further examine domain conditions under which fine-tuned SLMs can realistically replace or complement larger models, and where they still fall short, synthesizing recurring architectural motifs across telecom, medical, and cybersecurity settings.

The main contributions of this review are as follows: (1) It provides a synthesized overview of 54 studies covering cloud-based monolithic LLMs, hybrid SLM–LLM architectures, the performance of fine-tuned SLMs versus large models, and multi-agent workflows. (2) It reframes the literature around four distinct practical research questions: Performance, Efficiency, Architecture, and Domain Suitability. (3) It creates an analytical review matrix that reveals the commonalities and differences among the studies, mapping which specific research question each study addresses.

2 Related Works

Prior review literature on Small Language Models (SLMs) and SLM–LLM collaboration can be grouped into four highly relevant works that overlap with our scope at different levels of methodology and emphasis. The closest study is a recent PRISMA 2020-guided SLR that sources studies from multiple databases and provides a reproducible selection protocol [20]. While strong in transparency, it adopts a broader topical scope (SLMs in general), and its coverage window ends around early 2025, limiting visibility into the 2025–2026 wave of hybrid and agent-based systems.

The remaining three works are narrative surveys with substantial thematic overlap. One study provides wide coverage of SLM definitions, training techniques, applications, collaboration with LLMs, and trustworthiness, but does not follow a systematic and reproducible selection protocol [21]. Another presents a technically detailed taxonomy of collaboration patterns (e.g., routing, auxiliary modules, fusion, and distillation) with an emphasis on edge-device requirements, yet its analysis is primarily mechanism-centered rather than corpus-centered [22]. A further work overlaps strongly with our themes (performance, cost, privacy, and trustworthiness) but remains a narrative survey without a formal PRISMA-guided study selection and extraction schema [23].

Across these reviews, the gap is not a lack of attention to SLMs or collaboration, but a difference in analytical emphasis and evidential grounding. Existing surveys typically describe what kinds of models or mechanisms exist; however, they rarely align empirical evidence across domains or report comparable operational metrics. In contrast, our study is positioned as a PRISMA-guided, engineering-oriented synthesis that (i) treats domain suitability as a first-class axis, (ii) maps research questions to structured extraction fields to ensure traceability, and (iii) emphasizes quantitative indicators (accuracy, latency, cost, and energy) to support deployment-relevant decision-making.

3 Research Methodology

3.1 Research Questions

To systematically evaluate the technical and operational trade-offs between monolithic, hybrid, and multi-agent architectures, the review protocol is structured around the following four core research questions (RQs):

- **RQ1 (Performance):** How does the performance (accuracy/efficacy) of multi-agent SLMs or hybrid architectures (LLM + SLM) compare with that of single monolithic LLMs in specific reasoning and classification tasks?
- **RQ2 (Efficiency):** What is the impact of Hybrid and Multi-agent architectures on computational efficiency, specifically regarding inference cost, latency, and energy consumption, compared to Monolithic models?
- **RQ3 (Orchestration):** What orchestration and routing mechanisms are employed to manage the division of labor between large and small models in hybrid systems?
- **RQ4 (Domain Suitability):** In which specific domains (e.g., Healthcare, Coding, Mathematics) do SLMs achieve performance comparable to LLMs, and where do they fail (e.g., long-context reasoning)?

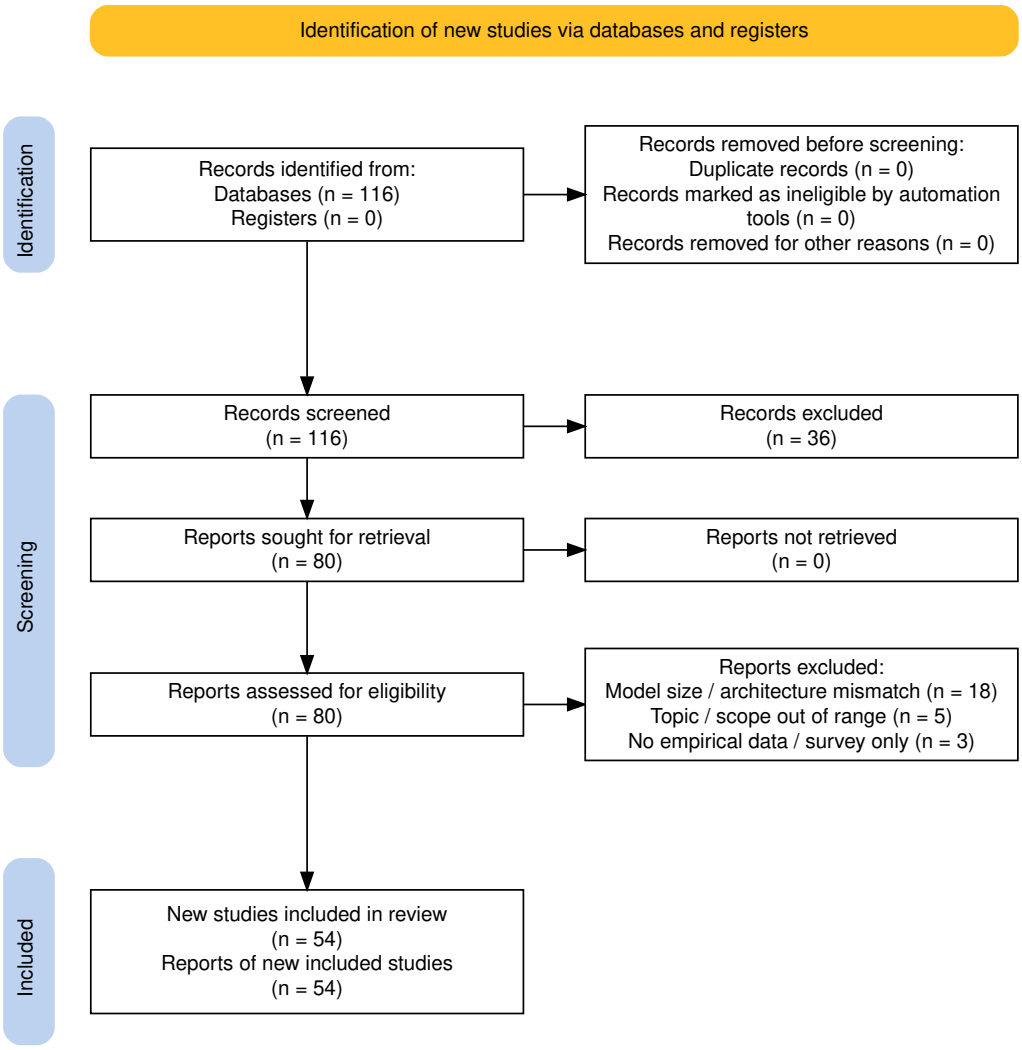

Fig. 1. PRISMA 2020 flow diagram for the systematic literature review. Generated using the PRISMA2020 Shiny App [24].

3.2 Information Sources and Search Strategy

To capture a comprehensive and high-quality corpus of contemporary language model architectures, the literature search was conducted utilizing the Web of Science (WoS) Core Collection. Utilizing WoS as the primary search engine allowed for the retrieval of indexed peer-reviewed journal articles, conference proceedings, and highly cited preprints within a single dataset.

The search strategy was systematically developed by extracting the core concepts from the defined research questions and refining the terminology through initial pilot searches. This iterative process helped identify common synonyms and alternate phrasing used in the literature (e.g., "Compact Model" for SLMs, or "Agentic" for multi-agent systems). To guarantee that every retrieved study examined the intersection of model size, architectural design, and system evaluation, the final query was structured into four mandatory components. The following Boolean query was applied to the title, abstract, and keyword fields:

> ("Large Language Model*" OR "LLM" OR "Generative AI" OR "Foundation Model*") AND ("Small Language Model*" OR "SLM" OR "Compact Model*" OR "Lightweight Model*") AND ("Multi-agent" OR "Hybrid" OR "Orchestration" OR "Routing" OR "Ensemble" OR "Agentic") AND ("Performance" OR "Accuracy" OR "Efficiency" OR "Latency" OR "Cost" OR "Benchmark")

3.3 Eligibility Criteria

To systematically isolate systems-level engineering research from the broader generative AI literature, retrieved records were evaluated against strict eligibility criteria. A comprehensive breakdown of the inclusion and exclusion parameters applied during the screening phases is presented in Table 1.

Table 1. Inclusion and Exclusion Criteria for Study Selection

Criteria	Inclusion (Include if...)	Exclusion (Exclude if...)
Architecture Scope	Studies evaluating SLMs alone or SLM and LLM collaborative architectures (e.g., Hybrid or Multi-agent).	Studies strictly comparing massive monolithic LLMs (e.g., GPT-4 vs. Gemini) without an SLM component.
Model Size	Architectures involving "small" models that strictly adhere to a parameter count of ≤ 7 Billion.	"Closed-box" or API-bound models where parameter count or architectural details are undisclosed (violates reproducibility).
Topic Focus	Focuses on quantitative system-level metrics such as performance, inference latency, memory, cost, or orchestration efficiency.	Focuses solely on ethical, sociological impacts, or legal regulations without technical benchmarks.
Data Quality	Contains empirical results and experimental validation (e.g., standard benchmarks or custom datasets).	Purely theoretical papers or non-empirical narrative surveys.

3.4 Study Selection Process

The study selection process strictly adhered to the PRISMA 2020 guidelines [25]. The progression of the complete screening funnel, detailing records identified, screened, sought, and assessed, is illustrated in Figure 1.

The initial literature search conducted via the Web of Science Core Collection yielded 116 records. As the dataset was generated from a single database, zero duplicate records required removal before the screening commenced.

The selection process was executed in two systematic phases. During the initial Identification and Screening phase, the title, abstract, and keywords of all 116 records were manually reviewed. At this stage, 36 records were excluded for failing to meet primary eligibility criteria, such as being non-technical narrative surveys, extended abstracts lacking full empirical validation, or operating in unrelated domains (e.g., computer vision pipeline optimization). Following this phase, 80 reports remained eligible and were carried forward to the in-depth full-text evaluation.

During the final Assessment phase, the 80 retrieved reports underwent a thorough full-text methodology review. This phase resulted in the exclusion of 26 additional records against specific technical constraints. The primary reasons for exclusion during full-text review were: (1) Model size or architecture mismatch (e.g., evaluated models exceeded the strict $\leq$ 7B parameter limit or lacked definitive SLM+LLM collaborative interaction) (n=18); (2) Topic or scope out of range (n=5); and (3) Lack of original empirical data or being a narrative survey not previously caught (n=3).

To ensure objectivity throughout the selection process, a team consensus mechanism was utilized. Inclusion and exclusion decisions at both stages were made via majority vote of the authors, with any conflicting tie resolved through collaborative discussion and escalation to the academic advisors. Following this rigorous selection pipeline, a final corpus of 54 primary studies was established for data extraction and synthesis.

3.5 Data Extraction and Synthesis Strategy

To systematically address the research questions, a standardized data extraction protocol was applied to the final corpus of 54 primary studies. For each study, data was extracted into a structured matrix to capture both the functional design of the systems and the empirical evidence used to justify them.

The extracted fields included the specific SLM and LLM models utilized; the overarching architecture type (e.g., Hybrid, Multi-Agent, Retrieval-Augmented Generation (RAG), Ensemble); orchestration mechanisms; evaluation metrics; baseline models; and quantitative performance indicators (accuracy, latency, cost, and energy). To ensure analytical rigor, each extracted variable was explicitly mapped to one or more of the core Research Questions (RQs). This mapping, together with concise field definitions, is detailed in Table 2.

Because the full comparative analysis table is provided in Appendix A, the mapping of Study IDs to their respective bibliographic citations is provided below for quick reference:

Primary Studies Mapping: S1: [6], S2: [13], S3: [8], S4: [19], S5: [26], S6: [18], S7: [27], S8: [7], S9: [28], S10: [11], S11: [29], S12: [12], S13: [30], S14: [31], S15: [32], S16: [33], S17: [17], S18: [34], S19: [35], S20: [36], S21: [37], S22: [38], S23: [39], S24: [40], S25: [41], S26: [42], S27: [15], S28: [43], S29: [44], S30: [19], S31: [45], S32: [10], S33: [46], S34: [4], S35: [47], S36: [48], S37: [49], S38: [50], S39: [51], S40: [52], S41: [1], S42: [2], S43: [9], S44: [53], S45: [54], S46: [55], S47: [56], S48: [5], S49: [3], S50: [57], S51: [58], S52: [59], S53: [14], S54: [60].

The comprehensive data extraction spreadsheet is also provided in Appendix A.

Table 2. RQ × data extraction matrix and field definitions used in the review. A check mark indicates that the field directly supports the corresponding research question.

Extraction Field	Description	RQ1	RQ2	RQ3	RQ4
SLM(s)	Specific small language model(s) evaluated, fine-tuned, or deployed.	✓			✓
LLM(s)	Large language model(s) used as a baseline, teacher, or fallback.	✓			✓
Architecture	Overall system design (e.g., hybrid, multi-agent, RAG, ensemble).	✓			✓
Orchestration Method	Coordination mechanism such as routing, decomposition, fusion, or voting.			✓	✓
Agent Count	Number of distinct agents, experts, or role-specific components.		✓		✓
Task Type	Downstream application, domain, or problem setting evaluated.	✓		✓	
Dataset	Benchmark, open-source corpus, or proprietary data used in evaluation.	✓		✓	
Metrics	Evaluation criteria used to assess performance and efficiency.	✓		✓	✓
LLM Baseline	Standalone performance score of the comparison large model.	✓		✓	✓
SLM Baseline	Standalone performance score of the comparison small model.	✓		✓	✓
Accuracy	Final performance score achieved by the proposed architecture.	✓		✓	
Latency (ms)	Reported inference delay, throughput, or time-to-response.		✓	✓	✓
Cost ($)	Reported API cost or computational savings versus full LLM use.		✓	✓	✓
Energy	Power use, joules-per-token, or carbon-related efficiency metric.		✓	✓	✓
Key Findings	Main study takeaway on SLM–LLM collaboration or orchestration.	✓	✓	✓	✓
Limitations	Reported constraints, failure modes, or deployment bottlenecks.		✓	✓	

Following data extraction, a narrative synthesis approach was utilized. Because the included studies operate across diverse domains (e.g., telecommunications, healthcare, software engineering) and report varying metrics, a purely statistical meta-analysis was not feasible. Instead, studies were grouped by their architectural taxonomy, allowing for a comparative analysis of how different orchestration methods and domain constraints impact the performance and efficiency trade-offs between large and small models.

4 Findings

The findings are organized below according to the four research questions defined in the review protocol.

4.1 RQ1: Comparative Performance of Hybrid and Multi-Agent Systems

The review shows that both hybrid and multi-agent architectures can outperform standalone small-model baselines, but their advantages emerge through different forms of coordination rather than parameter scaling alone. Hybrid systems achieve their strongest gains when computation is selectively distributed between local small models and stronger remote models. Token-level edge–cloud collaboration improves small-model performance without requiring full-query escalation: one study reports LLM-comparable quality on GSM8K at only 25.8% of full-cloud cost by using TinyLlama 1.1B or Llama-2-7B as edge-side drafting models and Llama-2-70B as a cloud verifier (S2). Similarly, routing only about 7% of low-confidence tokens from a 0.5B on-device model to a cloud-hosted Qwen2.5-32B-Instruct model yields over 60% performance improvement over the local-only setup while preserving more than 80% cost reduction relative to full LLM inference (S46). This pattern is reinforced by uncertainty-aware hybrid inference, where opportunistic skipping reduces uplink transmissions and LLM computations by 45.93%, reaches up to 97.54% of LLM inference accuracy, and delivers 2.54× faster token throughput than hybrid inference without skipping (S45). At the sub-task level, structured task decomposition and cloud–edge allocation reduce average reasoning time by 66.12% and API cost by 83.57% while maintaining reasoning accuracy comparable to the best baselines (S53). In domain-specific settings, hybrid orchestration can also outperform larger general-purpose models: a fine-tuned Phi-2 (2.7B) combined with retrieval, reranking, hybrid search, and long-context support achieves 84.20% accuracy, exceeding GPT-4o mini (82.05%) and GPT-4o (80.80%) on telecom question answering (S4).

Multi-agent systems, by contrast, show their comparative strength when performance depends on role specialization, critique, verification, and distributed reasoning. A three-agent Phi-3-based architecture performs on par with human experts in conceptual genomics tasks, although its gains weaken on more complex code-generation workflows, indicating that multi-agent benefits remain task-sensitive (S51). In reasoning-intensive question answering, a Proponent–Opponent–Arbitrator structure built on compact models such as Qwen2.5 0.5B and DeepSeek-R1 1.5B surpasses both single-agent reasoning and vanilla LLM baselines with only 1.8× inference-time overhead (S37). Likewise, assistant–checker collaboration supported by memory and feedback loops enables small-model multi-agent systems to approach or exceed larger baselines across several tasks: LeptoConnect-7B outperforms GPT-3.5-turbo by 163.1% on database tasks, while LeptoConnect-1.8B reaches 81.6% of GPT-4's score on knowledge graph tasks, and an enhanced TinyLlama-1.8B improves ROUGE-L by 63.4% over CodeLlama-7B in code correction (S20). Overall, the findings indicate that hybrid systems are most effective when the main challenge is balancing accuracy with latency and cost

through adaptive routing, whereas multi-agent systems are most effective when the task benefits from explicit decomposition of reasoning roles and iterative verification. Thus, the comparative performance advantage of both architecture families is driven primarily by orchestration strategy, not by model size alone.

4.2 RQ2: Efficiency, Cost, Latency, and Energy Trade-offs

The review reveals that while efficiency motivates the adoption of hybrid and multi-agent systems, the actual impact on cost, latency, and energy depends heavily on the orchestration method. Hybridization is not a universal efficiency solution; it introduces a trade-off space where computational bottlenecks are often shifted rather than eliminated.

Regarding inference cost and token consumption, architectures employing selective escalation or task decomposition drastically reduce reliance on massive standalone LLMs. S46 achieves an over 80% cost reduction compared to full LLM inference by routing fewer than 7% of generated tokens to the cloud. Similarly, S2 uses a token-level draft-verification method to match the quality of a 70B parameter LLM while consuming only 25.8% to 31.2% of standard cloud inference costs. Query-level decomposition yields similar savings: S53 cuts average API costs by 83.57% via parallel reasoning. Computational savings are also heavily emphasized through token reduction. S33 achieves a 95% reduction in LLM token transmissions via federated uncertainty-aware routing, and S27 eliminates redundant multi-round communications through an SLM-generated directed acyclic graph, reducing token consumption by 35% to 70%. In dialogue state tracking, S6 highlights that a retrieval-based router reduces overall computational FLOPs by approximately 62%.

Conversely, the impact on latency is highly polarized. In optimal scenarios, relying on lightweight models accelerates throughput. A fine-tuned SLM in a hybrid classification pipeline (S32) operates at 7.41 ms on a GPU, making it over 20 times faster than its larger counterpart. Furthermore, S45 shows that uncertainty-aware skipping in an edge-cloud setup provides 2.54× faster token throughput. However, orchestration mechanisms frequently introduce new latency bottlenecks. Planners and retrievers add non-trivial temporal overhead; the adaptive retrieval planner in S10 takes 1.42 seconds compared to 0.95 seconds for naive retrieval, and S4 explicitly identifies cross-encoder reranking as its primary latency bottleneck. Multi-agent collaboration also inherently multiplies inference cycles. S37 reports a 1.8× inference-time overhead for adversarial debate, and complex agentic workflows in S22 can push average response latencies to 107 seconds. Processing time is further impacted by hardware and modality constraints, such as constrained local hardware (S11), dual-path video sampling (S5), CPU-only zero-shot ensembles taking upwards of 859 seconds (S47), and multi-turn key-value cache recomputation at the edge (S48).

Energy consumption remains the least documented metric in the reviewed corpus, and most reported efficiency gains are resource proxies rather than direct energy measurements. The clearest energy-centered evidence comes from S34, which argues for energy-per-token as a more informative metric and shows that forcing a small model to execute complex reasoning workflows, such as Chain-of-Thought, can substantially increase its per-query energy use. In some settings, a larger model prompted more directly is therefore more energy-efficient than a smaller model that generates a long reasoning trace. By contrast, S20 reports a 26.4% reduction in computational redundancy through dual-scale memory, which may indicate lower resource demand but does not by itself constitute a direct energy benchmark. Taken together, the corpus suggests that energy efficiency depends on orchestration and generation length as much as on model size, and that stronger claims require standardized, hardware-specific measurement protocols.

4.3 RQ3: Orchestration and Routing Mechanisms

The studies contains a broad set of orchestration mechanisms, but they can be organized into a small number of recurring families. Across the included studies, performance differences are explained less by absolute model size than by how effectively the system retrieves evidence, routes uncertainty, and divides labor across components.

Token-level collaboration. Token-level orchestration is most visible in verification-oriented hybrid systems. S2 uses the SLM as a drafter and the LLM as a verifier on difficult tokens. S45 extends this pattern with uncertainty-aware skipping so that likely acceptable SLM tokens can bypass the remote model. S46 reaches a related design point by routing only low-confidence tokens for external inference. Taken together, these studies show that token-level collaboration provides fine-grained control over answer quality, but it can also introduce communication and synchronization overhead when escalation rates increase.

Query-level or example-level routing. Query-level routing appears when the system selects a processing path for the full input rather than for individual tokens. S6 exemplifies retrieval-based expert routing at the dialogue level, while S32 shows that instance-level model selection can also be effective in customer-review analysis. These designs are most useful when "easy" and "hard" cases can be separated in a semantically meaningful way before full generation begins.

Retrieval-centered orchestration. RAG is a major control mechanism in the corpus even when the architecture is classified as hybrid rather than purely RAG. S4, S10, S31, and S44 use retrieval not only to expand context but also to constrain the action space of a smaller model. In these studies, retrieval acts as a coordination layer that narrows ambiguity, grounds the answer in domain evidence, and reduces reliance on raw model scale alone.

Planner-based decomposition and agentic workflows. Task decomposition is increasingly important in the recent corpus. S53 uses a task decomposer, dependency graph, and plug-and-play adapter to allocate subtasks between local and remote models. S52 uses a feedback-integrated workflow that gathers structured feedback from multiple perspectives before revising the prompt of the smaller model. Related role-based decomposition also appears in S11, S20, and S51. Across these studies, the common idea is that small models remain viable when the workflow can be partitioned into specialist stages rather than handled as a single undifferentiated generation step.

Multimodal and heterogeneous fusion. Orchestration frequently extends beyond unimodal text processing to integrate diverse data modalities and specialized neural architectures. Several frameworks are designed to fuse language models with non-text representations and structured data. For example, S5 combines coarse-grained and fine-grained visual semantics through prompt-guided video sampling, while S3 fuses language-model embeddings with graph-convolutional features for molecular-property prediction. This heterogeneous routing is particularly prevalent in applied domains such as medicine; S17 orchestrates convolutional networks (ConvNeXt) for anatomical localization alongside a self-reflective LLM agent, and S25 employs a late-fusion approach to combine structured gradient boosting models (EHR tabular data) with unstructured clinical text analysis. In these settings, orchestration operates as a structured fusion mechanism, ensuring that visual, spatial, or mathematical data is processed by the optimal specialized architecture before language generation occurs.

4.4 RQ4: Domain Suitability and Application-Specific Constraints

The review reveals that the choice between hybrid and multi-agent systems is rarely domain-agnostic. The overarching constraints of the operational environment—whether they be strict privacy mandates, rigid latency budgets, or the need for iterative logical verification—dictate the orchestration method. Across the 54 primary studies, distinct architectural preferences emerge across four core applied domains.

Healthcare, Clinical, and Scientific Reasoning In fields with strict privacy constraints (e.g., HIPAA compliance) or high-stakes accuracy requirements, hybridization is driven by the need to keep data local while grounding the SLM in verifiable facts. Consequently, local SLMs fine-tuned on clinical corpora and augmented by RAG are the dominant architecture (S17, S21). For example, S17 presents a hybrid framework for medical report generation that successfully improved patient readability across 6th to 18th-grade levels without losing clinical rigor by maintaining local detection. When evaluating radiology reports, S12 demonstrated that few-shot prompting allowed open-source models to statistically match the performance of massive models like GPT-4, achieving a Micro F1 score of 0.98 versus 0.97. This reliance on local ensembles and structured tabular transformations extends heavily into diagnostic classifications, ranging from herbal formulas to cardiovascular outcome probabilities (S19, S24). Distillation also proves highly effective in this domain; S35 showed that distilling knowledge from a massive 30B parameter model into a lightweight SLM yields a highly accurate, privacy-preserving tool for multilingual medical summarization. In complex scientific domains, such as molecular property prediction, S3 demonstrated that a 1.5B parameter DeepSeek model fused with Graph Convolutional Networks (GCN) outperformed larger 7B and 8B variants, proving that local models can master highly specialized scientific vernacular when structurally augmented.

Software Engineering and Educational Assessment Tasks requiring rigid syntax, iterative refinement, and logical debugging benefit significantly from multi-agent or feedback-driven SLMs rather than zero-shot massive LLMs. In pedagogical text generation, S1 showed that a local 1.7B SLM orchestrated via a Markov Decision Process achieved a quality score of 4.17, which is directly comparable to the 4.15–4.31 range achieved by cloud-based LLMs. For code correction and database operations, S20 utilized specialized small models integrated with dual-scale memory systems to reduce computational redundancy by 26.4% and semantic mistakes by 34.8%. Automated assessment also benefits from structural orchestration, where highly structured report components allow smaller models to out-compete larger baselines (S23). Furthermore, orchestration frameworks frequently assign specific roles to SLMs to mimic the developer-reviewer dynamic and generate high-fidelity code offline (S13, S51, S52). S51 successfully utilized a three-agent system to generate bioinformatics code that approaches human-expert baselines, while S52 deployed a Feedback-Integrated Prompt Optimiser (FIPO) to iteratively revise SLM outputs based on explicit execution feedback. Finally, for code static analysis, S42 found that while massive LLMs demonstrate a clear advantage for broad type inference, smaller specialized models often suffice when integrated properly into the analysis pipeline.

Telecom and Networking Telecommunications architectures prioritize strict latency budgets, bandwidth allocation, and domain-specific vernacular, making hybrid edge-cloud systems the standard. The TeleOracle system evaluated in S4 demonstrated that an SLM generator could achieve an 81.20% accuracy in telecom domain QA—a 30% improvement over the base Phi-2 model—effectively

matching GPT-4 while keeping data within telecom network constraints. This is reinforced by S44, which found that combining LoRA fine-tuning with a highly accurate ColBERT retrieval mechanism allows small models to effectively parse and answer highly technical telecom jargon. Beyond text generation, latency optimization and bandwidth reduction are critical drivers for hybridization (S29, S45, S53). S29 integrated lightweight sequence predictions with reinforcement learning to enable proactive bandwidth scheduling, significantly outperforming traditional reactive methods. Similarly, S45 and S53 utilized uncertainty-aware speculative inference and sub-task decomposition to dynamically bypass cloud LLMs, drastically reducing uplink transmissions and cutting average API costs by up to 83.57%.

Edge Deployment, Industrial, and Cyber-Physical Systems Real-world interactions require physical hardware constraints, such as CPU-only limitations or embedded edge devices, to govern model orchestration. In hybrid edge-cloud setups for complex reasoning, S2 emphasizes that only approximately 3% of tokens are deemed "hard" enough to require cloud LLM verification, drastically saving edge compute resources. In mission-critical physical environments, distributed multi-agent architectures are relied upon to process raw operational data for geospatial and aerospace decision support (S11, S39). S11 highlights that locally embedded SLMs (e.g., Mistral 7B) can achieve accuracy comparable to cloud-based GPT-4o for specialized cockpit operations. In industrial video monitoring, S5 utilizes a dual-path hybrid framework to decouple visual information: the local SLM processes coarse-grained, uniform frames for rapid assessment, and only triggers the larger multimodal LLM to process keyframes when a complex, localized safety anomaly is detected. This modality-spanning orchestration extends into emerging cyber-physical tasks, such as cross-modal olfactory understanding in S41, where fine-tuned, heavily quantized multimodal SLMs process local sensor data to bridge the gap between physical inputs and language without remote computation.

4.5 Cross-RQ Synthesis

Viewed together, the four research questions suggest that the literature is converging toward a specific systems principle: hybrid language-model design works best when the large model is treated as a selectively invoked capability rather than the default engine for every token, query, or subtask.

The first implication is architectural. Strong results are associated with explicit control layers such as routers, planners, retrievers, or fusion modules. This means that future progress is unlikely to come only from making SLMs slightly larger or LLMs slightly cheaper. It will also depend on better confidence estimation, task decomposition, retrieval selection, and cross-model scheduling.

The second implication is methodological. Because accuracy dominates reporting, many studies overstate practical efficiency. The corpus already contains cases where stronger orchestration clearly reduces cost or latency, but it also contains cases where coordination shifts the bottleneck rather than removing it. A mature evaluation protocol should therefore report resource metrics as first-class outcomes rather than optional extras.

The third implication is application-oriented. The most successful SLM-centered deployments are not generic substitutes for LLMs. They are domain-adapted systems in which the small model is grounded, specialized, or protected by escalation logic. This is why telecom, structured reasoning, radiology labeling, optimisation formulation, and industrial monitoring are so prominent in the positive findings of the corpus.

4.6 Implications for Practitioners

For software engineers, system architects, and technical leads, the findings of this systematic review shift the discourse from theoretical model capabilities to practical system design. Because production environments are governed by strict non-functional requirements—such as latency budgets, API costs, and data privacy mandates—deploying a massive standalone LLM is rarely the optimal engineering choice. To bridge the gap between academic findings and industrial application, we synthesize the empirical evidence into an architectural decision matrix (Table 3).

Table 3. Architectural Decision Matrix for Practitioners

Operational Constraint	Recommended Architecture	Target Applications	Key Engineering Considerations
Strict Data Privacy & Air-Gapped Networks	Local SLM + RAG (Unimodal)	Healthcare, Telecom, Aerospace, Defense (e.g., S4, S11, S17, S21)	Zero API cost and total data sovereignty. High reliance on retriever quality. Bottleneck shifts to local memory and hardware capacity.
High Throughput & Low Inference Budget	Hybrid (Edge-Cloud) with Uncertainty Routing	IoT Data Processing, Network Traffic, Routing (e.g., S29, S33, S45, S46)	Drastically reduces API costs. Introduces network synchronization latency. Requires robust threshold tuning to decide when to escalate tokens to the cloud.
Complex Logical Reasoning & Verification	Multi-Agent System (SLM Ensembles)	Automated Assessment, Code Generation, Math (e.g., S20, S27, S37, S51)	High determinism and accuracy. Significant inference latency penalty due to multiple generation/debate passes. Requires rigid task decomposition.
Sensor & Physical Data Integration	Heterogeneous Fusion (SLM + Specialized Models)	Cyber-Physical Systems, Medical Imaging, Robotics (e.g., S3, S5, S17, S41)	Requires late-fusion orchestration. Overall system latency is dictated by the slowest specialized model (e.g., CNN or GCN) in the pipeline.

Beyond model selection, the corpus highlights three foundational principles for engineering applied language systems:

Design for Routing, Not Just Generation. The most critical component in modern LLM/SLM systems is the orchestration layer, not the generator itself. Practitioners should invest engineering

effort into building robust semantic routers, uncertainty estimators, and task decomposers. As demonstrated across the hybrid architectures in this review, routing only 3% (S2) to 7% (S46) of the most difficult tokens or queries to a massive cloud model can yield performance statistically identical to fully cloud-based solutions, but at a fraction of the cost.

Acknowledge the Latency-Cost Trade-off. Efficiency is not a monolith. While multi-agent systems and agentic workflows drastically reduce reliance on expensive LLM APIs (optimizing cost), they inherently multiply inference cycles (sacrificing latency, as seen in S22 and S37). System architects must define explicit Service Level Agreements (SLAs). If real-time responsiveness is required—such as telecom routing (S4) or active cockpit support (S11)—token-level hybrid skipping (S45) or highly optimized standalone SLMs are mandatory. If the task is asynchronous—such as offline code generation (S13) or batch medical report analysis (S12)—multi-agent SLM debates or self-reflective loops (S17, S37) provide the best return on investment.

Treat RAG as a Control Mechanism, Not Just a Search Tool. In the context of Small Language Models, Retrieval-Augmented Generation is not merely a way to inject real-time knowledge; it is a critical security and bounding mechanism. Because SLMs lack the parameter capacity to store vast amounts of world knowledge, their success is almost entirely bottlenecked by the quality of the external retriever (S44). Practitioners must utilize RAG as a rigid coordination layer that constrains the SLM's action space and grounds its outputs in verifiable facts (S4, S10, S31), transforming a general-purpose model into a reliable, domain-specific worker.

5 Threats to Validity

This review has several threats to validity that should be stated explicitly.

Internal validity. Although the review follows a PRISMA-guided workflow, study identification, screening, and extraction still involve researcher judgment. Borderline papers may have been interpreted differently under another reading of the inclusion criteria, especially in cases where the SLM component or the empirical contribution was not equally central throughout the paper. The representative PDF inspection used to deepen the narrative also adds a second judgment layer, because not every included study was read in the same depth. To reduce this risk, borderline screening decisions were discussed through team consensus, and the synthesis remained tied to the predefined research questions and the same extraction matrix throughout.

Construct validity. Some of the core constructs in this review are not standardized across the literature. In particular, "SLM", "hybrid architecture", "multi-agent system", and "efficiency" are defined differently from paper to paper. A single hybrid label may refer to token-level verification, retrieval-grounded generation, planner-based decomposition, or multimodal fusion. Likewise, efficiency may mean latency, FLOPs, API cost, bandwidth reduction, or energy use. This threatens direct comparability and means that some studies grouped under the same high-level category differ substantially in mechanism. To partially limit this, the review separates broad architecture labels from extracted orchestration mechanisms and discusses recurring mechanism families explicitly.

Conclusion validity. The strongest limitation for cross-study inference is the uneven and inconsistent reporting of evaluation outcomes. Accuracy is reported in most papers, but latency, cost, and energy are available only in a minority, often with different units, hardware assumptions, or deployment settings. For that reason, the review can identify recurring patterns, but it cannot support a formal meta-analysis or strong quantitative claims about the size of efficiency gains across

the whole corpus. This threat is reinforced by publication and reporting bias, since positive hybrid results are more likely to be published and summarized than negative or inconclusive findings. To partly temper this, the extraction also recorded reported limitations, and the synthesis emphasizes uneven reporting and task-sensitive trade-offs rather than universal gains.

External validity. The corpus is based on a Web of Science-centered search and a bounded review window. This improves transparency and reproducibility, but it may miss relevant studies from other databases, preprint platforms, or fast-moving venues. At the same time, WoS provides broad coverage across major peer-reviewed journals and conference proceedings in a single indexed source, which helps reduce, though not remove, this coverage risk. The field also evolves quickly, and the current corpus is heavily concentrated in 2025 and in a limited set of application domains. Therefore, the conclusions should be generalized cautiously beyond the covered time period, tasks, and deployment settings.

6 Conclusion

This paper presented a systematic literature review of 54 studies on SLMs, LLMs, and SLM–LLM collaboration across monolithic, hybrid, multi-agent, ensemble, and retrieval-centered architectures. The review was structured around four practical questions: comparative performance, efficiency, orchestration, and domain suitability. Across these four axes, the main pattern is consistent: SLMs are most competitive when they are supported by retrieval, routing, uncertainty estimation, task decomposition, prompt optimisation, or multimodal fusion rather than used as isolated replacements for larger models.

The findings also show that the literature is stronger in architectural innovation than in evaluation discipline. Many studies report accuracy improvements, but far fewer report latency, cost, or energy in a form that supports fair comparison. As a result, the current evidence is sufficient to show that hybridization can work, but often insufficient to show when it is the better deployment choice under realistic resource constraints. The review also suggests that the strongest positive results are concentrated in narrower or more structured settings such as telecom question answering, selected healthcare tasks, optimisation, and industrial monitoring, while open-ended reasoning and broad general-purpose use remain less convincing for SLM-centered systems.

The limited energy-related evidence in the reviewed literature supports a narrower conclusion than broad efficiency claims often imply. Direct energy measurement is rare, and the clearest energy-centered result shows that when small models are pushed into long Chain-of-Thought traces, their per-query energy use can exceed that of a larger model prompted more directly [4]. Other studies in the corpus report only proxy improvements, such as a 26.4% reduction in computational redundancy through dual-scale memory [36], which may indicate lower resource demand but not verified energy savings. Future work should therefore treat energy as a first-class evaluation outcome and report energy-per-token, reasoning length, hardware configuration, and escalation behavior under comparable conditions. In that sense, the open question is not whether SLM-centered systems can be efficient in principle, but when their orchestration actually translates into measured energy savings in practice.

References

1. Esteban-Romero, S., Martín-Fernández, I., Gil-Martín, M., Fernández-Martínez, F.: Synthesizing Olfactory Understanding: Multimodal Language Models for Image-Text Smell Matching. Symmetry **17**(8),

1021 (2025)
2. Shivarpatna Venkatesh, A.P., Sunil, R., Sabu, S., Reis, M.S., Bodden, E., Amir, M.: An Empirical Study of Large Language Models for Type and Call Graph Analysis in Python and JavaScript. arXiv preprint arXiv:2410.00603 (2025)
3. Chen, H., Deng, W., Yang, S., Xu, J., Jiang, Z., Ngai, E.C.H., Liu, J., Liu, X.: Toward Edge General Intelligence via Large Language Models: Opportunities and Challenges. arXiv preprint arXiv:2410.18125 (2025)
4. Wilhelm, P., Wittkopp, T., Kao, O.: Beyond Test-Time Compute Strategies: Advocating Energy-per-Token in LLM Inference. In: Proc. 5th Workshop on Machine Learning and Systems (EuroMLSys '25), pp. 1–7. ACM (2025)
5. Bao, R., Xue, N., Sun, Y., Chen, Z.: Dynamic Quality-Latency Aware Routing for LLM Inference in Wireless Edge-Device Networks. arXiv preprint arXiv:2508.11291 (2025)
6. Tigani, S., et al.: CPU-Only Self Enhancing Authoring Copilot Design-Based Markov Decision Processes Orchestration and Qwen 3 Local LLM. Economies **13**(11), 520 (2025)
7. Aydın, S.K., Ali, R.H., Faiz, S., Khan, T.A.: An Integrated AI Framework for Personalized Nutrition Using Machine Learning and Natural Language Processing for Dietary Recommendations. Appl. Sci. **15**(17), 9283 (2025). doi: 10.3390/app15179283
8. Xie, L., Jin, Y., Xu, L., Chang, S., Xu, X.: Fusing Domain Knowledge with a Fine-Tuned Large Language Model for Enhanced Molecular Property Prediction. J. Chem. Theory Comput. **21**(14), 6743–6758 (2025). doi: 10.1021/acs.jctc.5c00605
9. Zhu, X., Li, J., Liu, Y., Ma, C., Wang, W.: Distilling Mathematical Reasoning Capabilities into Small Language Models. arXiv preprint arXiv:2401.11864 (2024)
10. Alabbasi, N., Alhussein, O., Muhaidat, S., Debbah, M.: Do you actually need an LLM? Rethinking language models for customer reviews analysis. arXiv preprint arXiv:2411.02555 (2024)
11. Ji, X., Chen, S., Shon, Y., et al.: RAP-RAG: A Retrieval-Augmented Generation Framework with Adaptive Retrieval Task Planning. Electronics **14**(1), 185 (2025)
12. Dorfner, F.J., Gadewar, M.R., et al.: Comparing Commercial and Open-Source Large Language Models for Labeling Chest Radiograph Reports. Radiology **313**(1), e240590 (2024)
13. Hao, Z., Wei, J., Wang, T., Ren, J.: Hybrid SLM and LLM for Edge-Cloud Collaborative Inference. In: Proc. of the 2024 Workshop on Edge and Mobile Foundation Models (EdgeFM '24). ACM, New York, pp. 1–7 (2024)
14. Shao, C., Hu, X., Lin, Y., Xu, F.: Division-of-Thoughts: Harnessing Hybrid Language Model Synergy for Efficient On-Device Agents. In: Proc. of the ACM Web Conference 2025 (WWW '25), pp. 1822–1833. ACM (2025)
15. Zeng, J., Liao, J., Sun, S., Zhang, G., Xu, C., Liu, H., Li, Y.: Multi-Agent Collaborative Reasoning via Cloud-Edge Framework. IEEE Network (2026). DOI: 10.1109/MNET.2026.3651036
16. Chatzistefanidis, I., Nikaein, N.: Symbiotic Agents: A Novel Paradigm for Trustworthy AGI-driven Networks. Comput. Networks **273**, 111749 (2025)
17. Han, J., Kim, M., Kim, Y., Lee, W.H.: Self-Reflective Chest X-Ray Report Generation with Clinical-Aware Detection and Multilevel Readability. Journal of Medical Systems **49**, 190 (2025)
18. Lee, C.H., Cheng, H., Ostendorf, M.: OrchestraLLM: Efficient Orchestration of Language Models for Dialogue State Tracking. In: Proc. of the 2024 Conf. of the North American Chapter of the Association for Computational Linguistics (NAACL-2024). ACL (2024)
19. Alabbasi, N., Erak, O., Alhussein, O., Lotfi, I., Muhaidat, S., Debbah, M.: TeleOracle: Fine-Tuned Retrieval-Augmented Generation with Long-Context Support for Networks. arXiv preprint arXiv:2411.02617 (2024)
20. Author(s): State of the Art and Future Directions of Small Language Models: A Systematic Review. Big Data & Cognitive Computing (2025)
21. Wang, F., et al.: A Comprehensive Survey of Small Language Models in the Era of Large Language Models: Techniques, Enhancements, Applications, Collaboration with LLMs, and Trustworthiness. arXiv:2411.03350 (2024)

22. Chen, Y., et al.: A Survey on Collaborative Mechanisms Between Large and Small Language Models. arXiv:2505.07460 (2025)
23. Wang, L., et al.: A Survey on Collaborating Small and Large Language Models for Performance, Cost-effectiveness, Cloud-edge Privacy, and Trustworthiness. arXiv:2510.13890 (2025)
24. Haddaway, N. R., et al.: PRISMA2020: An R package and Shiny app for producing PRISMA 2020-compliant flow diagrams. Campbell Systematic Reviews **18**, e1230 (2022). https://doi.org/10.1002/cl2.1230
25. Page, M.J., McKenzie, J.E., Bossuyt, P.M., Boutron, I., Hoffmann, T.C., Mulrow, C.D., et al.: The PRISMA 2020 statement: an updated guideline for reporting systematic reviews. BMJ **372**, n71 (2021)
26. Cao, Q., et al.: Toward Energy-Safe Industrial Monitoring: A Hybrid LLM Framework for Video Captioning. Appl. Sci. **15**(23), 12848 (2025)
27. Han, X., Sun, J., Yu, S., Qin, L., Luo, H., Li, Y.: CommentAgent: a LLM-powered agent framework for automated comment generation and opinion understanding. J. Supercomput. **81**(15), 1438 (2025). doi: 10.1007/s11227-025-07957-6
28. Demir, E., Bilgin, M.: DPATransLLM: Detection of Pronominal Anaphora in Turkish Sentences Using Transformer-Based, Large Language Models and Hybrid Ensemble Approach. Appl. Sci. **15**(23), 12480 (2025). doi: 10.3390/app152312480
29. Pacheco De Almeida Prado, J., de Oliveira, A.J.S., et al.: Enhancing Flight Deck Decision Support with Distributed GenAI: A Multi-Agent Approach. In: Proc. 2025 IEEE Aerospace Conf. (AERO), pp. 1–12. IEEE (2025)
30. Zhu, J., et al.: OFSLMs: Offline Fine-Tuned SLMs based on Hybrid Synthetic Knowledge for Robot Introspective Decision-making. IEEE ROBIO (2024)
31. Jeong, S., et al.: Test-Time Self-Adaptive Small Language Models for Question Answering. EMNLP (2023)
32. Weiss, Y., Buttery, P., Africa, D.D., Diehl Martinez, R.: Investigating ReLoRA: Effects on the Learning Dynamics of Small Language Models. arXiv preprint arXiv:2509.12960 (2025)
33. Chatzistefanidis, G., et al.: Symbiotic Large and Small Language Models: A Survey. arXiv:2501.00000 (2025)
34. Pietrzak, Z., Mączka, K., Niemiec, M.: What if, Behind the Curtain, There Is Only an LLM? A Holistic Evaluation of TinyLlama-Generated Synthetic Cyber Threat Intelligence. Electronics **14**, 4971 (2025)
35. Gao, Y., Myers, S., Chen, S., Dligach, D., Miller, T., Bitterman, D.S., et al.: Uncertainty estimation in diagnosis generation from large language models: next-word probability is not pre-test probability. JAMIA Open **8**(1), ooae154 (2025)
36. Xiong, Y., Huang, M., Liang, X., Tao, M.: ResilioMate: A Resilient Multi-Agent Task Executing Framework for Enhancing Small Language Models. IEEE Access **13**, 86892–86911 (2025)
37. Ahmed, M., Arefeen, M.A., Zobaed, S.M., Rouf, M.T., Saha, S., Rahman, M.S.I.: Adapting Large Language Models for Arabic: Comparative Evaluation, Fine-Tuning, and Ethical Deployment. IEEE Access **13**, 12902–12928 (2025)
38. Rahman, M.S.I., Arefeen, M.A., Rouf, M.T., Ahmed, M., Zobaed, S.M., Saha, S., Samanta, S.: Prompt Design for Medical Question Answering with Large Language Models. IEEE Access **12**, 168239–168266 (2024)
39. Abbas, N., Atwell, E.: Cognitive Computing with Large Language Models for Student Assessment Feedback. Big Data Cogn. Comput. **9**(5), 112 (2025)
40. Han, X., Han, C., Chen, Y., Wei, J.: A Weighted Voting Approach for Traditional Chinese Medicine Formula Classification Using Large Language Models: Algorithm Development and Validation Study. JMIR Med. Inform. **13**, e60787 (2025)
41. Dodson, R.J.B., Dodson, C.R.S., Dodson, P.S., et al.: Explainable machine learning-based prediction of psoriatic arthritis flares using heterogenous real-world data for personalised patient care. Rheumatology **65**(2), 528–537 (2026)
42. Liu, X., Myers, S., Bitterman, D.S., Dligach, D., Miller, T., Afshar, M., Gao, Y.: Improving VTE Identification through Language Models from Radiology Reports: A Comparative Study of Mamba, Phi-3 Mini, and BERT. JAMIA Open **8**(1), ooae155 (2025)

43. Krishna, S.V., Saini, A.N.: Compos Mentis at SemEval-2024 Task 6: A Multi-Faceted Role-based Large Language Model Ensemble to Detect Hallucination. In: Proc. SemEval-2024, pp. 930–944. ACL (2024)
44. Chen, Y., Ma, S., Wang, S., Shi, J.: Integrating Reinforcement Learning and LLM with Self-Optimization Network System. IEEE Commun. Lett. **29**(1), 228–232 (2025)
45. Miller, K., Ghosh, S., Neumann, F.: Towards Optimizing a Retrieval Augmented Generation using Large Language Model on Academic Data. Bachelor Thesis, University of Adelaide (2024)
46. Solat, F., Lee, J., Seif, M., Niyato, D., Poor, H.V.: Federated Learning-Enabled Hybrid Language Models for Communication-Efficient Token Transmission. IEEE Internet Things J. **12**(24), 53574–53587 (2025)
47. Lee, C., Kumar, S., Vogt, K.A., Munshi, M., Tallapudi, P., Vogt, A., Awad, H., Khan, W.: Democratizing cost-effective, agentic artificial intelligence to multilingual medical summarization through knowledge distillation. Sci. Rep. **15**, 27619 (2025)
48. Jin, X., Wang, Z., Chen, J., Yang, L., Oh, B., Hwang, S., Li, J.: HLMEA: Unsupervised Entity Alignment Based on Hybrid Language Models. In: Proc. 39th AAAI Conf. on Artificial Intelligence (AAAI-25). AAAI (2025)
49. Huynh, T., Nguyen, D.: Collaborating Agents with Large Language Models for Question-Answering Tasks. In: Proc. 5th IEEE Int. Conf. on Software Engineering and Artificial Intelligence (SEAI), pp. 1–7. IEEE (2025)
50. Al Daoud, E., Al Daoud, L., Asassfeh, M., Al-Shaikh, A., Al-Sherideh, A.S., Afaneh, S.: Enhancing Cybersecurity with Transformers: Preventing Phishing Emails and Social Media Scams. In: Proc. IEEE Conf. on Dependable and Secure Computing (DSC), pp. 1–8. IEEE (2024)
51. Ying, S., Li, Z., Yu, M.: Beyond Words: Evaluating Large Language Models in Transportation Planning. arXiv preprint arXiv:2408.01633 (2024)
52. Pintelas, E., Koursaris, A., Livieris, I.E., Tampakas, V.: TextNeX: Text Network of eXperts for Robust Text Classification—Case Study on Machine-Generated-Text Detection. Mathematics **13**(10), 1555 (2025)
53. Gichamba, A., Idris, T.K., Ebiyau, B., Nyberg, E., Mitamura, T.: ColBERT Retrieval and Ensemble Response Scoring for Language Model Question Answering. arXiv preprint arXiv:2408.10808 (2024)
54. Oh, S., Kim, J., Park, J., Ko, S.W., Quek, T.Q.S., Kim, S.L.: Uncertainty-Aware Hybrid Inference with On-Device Small and Remote Large Language Models. In: Proc. 2025 IEEE Int. Conf. on Machine Learning for Communication and Networking (ICMLCN). IEEE (2025)
55. She, J., Zheng, W., Liu, Z., Wang, H., Xing, E., Yao, H., Ho, Q.: Token Level Routing Inference System for Edge Devices. arXiv preprint arXiv:2504.07878 (2025)
56. Cielen, D., De Bock, K.W., Florès, L.: Exploring Zero-Shot SLM ensembles as an alternative to LLMs for sentiment analysis. Information Fusion **126**, 103666 (2026)
57. Li, M., Jiao, H., Zhou, T., Zhang, N., Peters, S., Lissitz, R.W.: Item Difficulty Modeling Using Fine-tuned Small and Large Language Models. In: Proc. 19th Workshop on Innovative Use of NLP for Building Educational Applications (BEA 2024), pp. 1–10 (2025)
58. Mehandru, N., Hall, A.K., Melnichenko, O., Dubinina, Y., Tsirulnikov, D., Bamman, D., Alaa, A., Saponas, S., Malladi, V.S.: BioAgents: Bridging the gap in bioinformatics analysis with multi-agent systems. Sci. Rep. **15**, 39036 (2025)
59. Amarasinghe, P.T., Nguyen, S., Sun, Y., Alahakoon, D.: Feedback-integrated prompt optimiser for problem formulation. Sci. Rep. **15**, 43543 (2025)
60. Li, C., Chen, Q., Li, L., Wang, C., Li, Y., Chen, Z., Zhang, Y.: Mixed Distillation Helps Smaller Language Model Better Reasoning. arXiv preprint arXiv:2312.10730 (2024)

A Online Appendix

The online appendix contains the full comparative analysis table for the 54 primary studies and the comprehensive data extraction spreadsheet used in this review.
Access link: https://bit.ly/SLM-LLM-Data-2026